

ЛАБОРАТОРНАЯ РАБОТА № 1

Язык SQL. Манипулирование базой данных. Реляционная алгебра и язык SQL

1.1 Цель работы

Научиться анализировать предметную область с целью создания схемы БД, учитывая ссылочную целостность набора. Изучить основы реляционной алгебры, как базового средства манипулирования. Выработать у обучающихся практические навыки по работе с реляционными базами данных и представлению запросов как на языке реляционной алгебре, так и SQL.

1.2 Основные положения

1.2.1 Создание схемы базы данных. Ссылочная целостность

Свое название реляционные базы данных получили именно по той причине, что таблицы в БД не существуют независимо друг от друга. Таблицы взаимосвязаны друг с другом, т.е. действие, произведенное в одной таблице, вызовет некоторые действия в другой таблице.

Существует три основных класса связей между таблицами:

- один к одному (1:1);
- один ко многим (1:M);
- многие ко многим (M:M).

На практике связи первого типа используются редко. Связи третьего типа не реализуются в РБД напрямую, одну связь многие ко многим приводят к двум связям один ко многим. Поэтому связь 1:M используется наиболее часто, ее следует рассмотреть наиболее подробно.

Кроме арности связи и ограничений целостности различают также идентифицирующие и не идентифицирующие связи.

С помощью идентифицирующей связи устанавливается взаимосвязь между зависимыми сущностями (клиент и его покупки). **Не идентифицирующая связь устанавливает взаимосвязь между независимыми сущностями** (автомобиль и его цвет). Разница между идентифицирующей и не идентифицирующей связью состоит в том, что при идентифицирующей связи внешний ключ является частью первичного ключа дочерней сущности, а при не идентифицирующей связи внешний ключ не входит в первичный ключ дочерней сущности. На ER-диаграмме идентифицирующая связь изображается непрерывной линией, не идентифицирующая – пунктирной.

Существуют четыре вида ограничений:

- первичный ключ (PRIMARY KEY);
- уникальный ключ (UNIQUE);
- внешний ключ (REFERENCES или FOREIGN KEY);
- проверочное ограничение (CHECK).

```
...
, [CONSTRAINT <constraint_name>] {
    PRIMARY KEY (<domain1>[, <domainN >]) [USING [ASC|DESC] INDEX <index_name>] |
    FOREIGN KEY (<domain1>[, <domainN >])
        REFERENCES <table_name > (<domain1 > [, <domainN >])
        [ON UPDATE <rule >]
        [ON DELETE <rule >] [USING INDEX <index_name>] |
    UNIQUE (<domain1 >[, <domainN >]) [USING [ASC|DESC] INDEX <index_name >] |
    CHECK <check_condition>
}
...
```

Ограничения могут быть указаны *на уровне столбца* (ограничения столбцов) или *на уровне таблицы* (табличные ограничения). Ограничения уровня таблицы необходимы, когда

ключи (ограничение уникальности, первичный ключ или внешний ключ) должны быть сформированы по нескольким столбцам, или, когда ограничение CHECK включает несколько столбцов, т.е. действует на уровне записи. Синтаксис для некоторых типов ограничений может незначительно отличаться в зависимости от того определяется ограничение на уровне столбца или на уровне таблицы.

Ограничение на уровне столбца указывается после определения других характеристик столбца и может включать только столбец, указанный в этом определении.

Ограничения на уровне таблицы указываются после определений всех столбцов. Ограничения таблицы являются более универсальным способом записи ограничений, поскольку позволяют ограничение более чем для одного столбца таблицы.

Можно смешивать ограничения столбцов и ограничения таблиц в одном операторе CREATE TABLE.

Существует два *типа ограничений*:

- *именованные* – необязательное предложение CONSTRAINT задаёт имя ограничения;
- *автоматически создаваемые индексы* – для первичного ключа (PRIMARY KEY), уникального ключа (UNIQUE) и внешнего ключа (REFERENCES) система автоматически создаёт соответствующий индекс.

Если задано имя ограничения, то автоматически создаваемому индексу будет присвоено это имя. Если имя ограничения не задано, то ограничению будет присвоено имя, автоматически генерированное СУБД. Для любого ограничения можно указать имя.

Предложение USING позволяет задать определённое пользователем имя автоматически создаваемого индекса, и опционально определить, какой это будет индекс – по возрастанию (по умолчанию) или по убыванию.

Первичный ключ (PRIMARY KEY, PK). Ограничение первичного ключа PRIMARY KEY строится на поле с заданным ограничением NOT NULL и требует уникальности значений столбца. Таблица может иметь только один первичный ключ. Первичный ключ по единственному столбцу может быть определён как на уровне столбца, так и на уровне таблицы. Первичный ключ по нескольким столбцам может быть определён только на уровне таблицы.

```
CREATE TABLE COUNTRY (
  COUNTRY COUNTRYNAME NOT NULL PRIMARY KEY,
  CURRENCY VARCHAR(10) NOT NULL);
```

или (рекомендуемый вариант, с указанием имени CONSTRAINT)

```
CREATE TABLE MY_TABLE (
  ID INTEGER NOT NULL,
  SOME_ID INTEGER NOT NULL,
  .....
  CONSTRAINT PK_MY_TABLE PRIMARY KEY(ID)
);
```

Внешний ключ (FOREIGN KEY, FK)

```
FOREIGN KEY ( < domain1 > [, < domainN >] )
  REFERENCES < TABLE_NAME > [ < Rdomain1 > [, < RdomainN >] ]
  [ON DELETE < rule > ] [ON UPDATE < rule > ];
```

где

```
< rule > = {NO ACTION | CASCADE | SET DEFAULT | SET NULL };
```

На уровне столбца ограничение внешнего ключа определяется предложением REFERENCES. После ключевого слова REFERENCES указывается имя родительской таблицы, на первичный или уникальный ключ которой ссылается описываемый внешний ключ. Имя столбца родительской таблицы, являющегося первичным (уникальным) ключом, помещается сразу после имени таблицы и заключается в круглые скобки, например

```
... ,
ARTIFACT_ID INTEGER REFERENCES COLLECTION(ARTIFACT_ID) ,
...
```

Синтаксис определения внешнего ключа на уровне таблицы несколько отличается. После определения всех столбцов, с их ограничения уровня столбца, вы можете определить именованное ограничение внешнего ключа уровня таблицы, используя ключевые слова **FOREIGN KEY** и имён столбцов для которых оно применяется:

```
... ,
CONSTRAINT FK_ARTSOURCE FOREIGN KEY(DEALER_ID, COUNTRY)
REFERENCES DEALER (DEALER_ID, COUNTRY) ,
```

Внешний ключ не требует ограничения **NOT NULL** для столбца или столбцов, на которые ссылается внешний ключ, хотя оно будет, если внешний ключ ссылается на столбец или столбцы, входящие в первичный ключ родительской таблицы. Тем не менее, внешний ключ может ссылаться на столбец или столбцы, входящие в уникальный ключ. **NULL** допускается в уникальных ключах, и, с некоторыми ограничениями, в уникальных ключах, построенных на нескольких столбцах.

Для обеспечения дополнительной целостности данных можно указать необязательные опции **ON DELETE** и **ON UPDATE**, которые обеспечат согласованность данных между родительскими и дочерними таблицами по заданным правилам:

- предложение **ON UPDATE** определяет, что произойдёт с записями подчинённой таблицы при изменении значения первичного/уникального ключа в строке главной таблицы;
- предложение **ON DELETE** определяет, что произойдёт с записями подчинённой таблицы при удалении соответствующей строки главной таблицы.

Правила удаления управляют тем, что будет происходить в базе данных при удалении строки. Аналогично правила вставки и обновления управляют тем, что будет происходить с базой данных, если строки изменяются или добавляются.

Для обеспечения ссылочной целостности внешнего ключа, когда изменяется или удаляется значение связанного первичного или уникального ключа, могут быть выполнены следующие действия:

- **NO ACTION** – не будет выполнено никаких действий; в клиентской программе должны быть предприняты специальные меры по поддержанию ссылочной целостности данных, иначе обновление/удаление не будет произведено и будет выдано соответствующее сообщение об ошибке;
- **CASCADE** – при изменении или удалении значения первичного ключа над значением внешнего ключа будут произведены те же действия. При выполнении удаления строки в главной таблице в подчинённой таблице должны быть удалены все записи, имеющие те же значения внешнего ключа, что и значение первичного (уникального) ключа удалённой строки главной таблицы. При выполнении обновления записи главной таблицы в подчинённой таблице должны быть изменены все значения внешнего ключа, имеющие те же значения, что и значение первичного (уникального) ключа изменяемой строки главной таблицы. Например, вместе с филиалом фирмы будут удаляться все агенты данного филиала.;
- **SET DEFAULT** – значения внешнего ключа всех соответствующих строк в подчинённой таблице устанавливаются в значение по умолчанию, заданное в предложении **DEFAULT** для этого столбца. Например, при удалении филиала фирмы все агенты могут быть переведены в другой филиал;
- **SET NULL** – значения внешнего ключа всех соответствующих строк в подчинённой таблице устанавливаются в пустое значение **NULL**;
- **RESTRICT** – выдать ошибку, показывающую, что при удалении или изменении записи произойдёт нарушение ограничения внешнего ключа. Этот вариант подобен **NO ACTION**, но эта проверка будет не откладываемой.

```
CREATE TABLE JOB (
    JOB_CODE          JOBCODE NOT NULL,
```

```

JOB_GRADE          JOBGRADE NOT NULL,
JOB_COUNTRY        COUNTRYNAME,
MIN_SALARY         NUMERIC(18,2) DEFAULT 0 NOT NULL,
MAX_SALARY         NUMERIC(18,2) NOT NULL,
LANGUAGE_REQ       VARCHAR(15) [1:5],
PRIMARY KEY (JOB_CODE, JOB_GRADE, JOB_COUNTRY),
FOREIGN KEY (JOB_COUNTRY) REFERENCES COUNTRY(COUNTRY)
ON UPDATE CASCADE
ON DELETE SET NULL,
CONSTRAINT CHK_SALARY CHECK(MIN_SALARY < MAX_SALARY)
или

```

```

ALTER TABLE MY_TABLE
ADD CONSTRAINT FK_MY_TABLE_SOME_ID FOREIGN KEY(SOME_ID)
REFERENCES SOME_TABLE(ID)
ON DELETE CASCADE
ON UPDATE CASCADE;

```

Ограничение уникальности (UNIQUE). Уникальный ключ – дополнительная возможность ограничения значений записей таблицы с целью профилактики занесения в нее двух и более записей, имеющих одинаковое значение указанного столбца (столбцов). В отличие от PRIMARY KEY количество уникальных ключей в таблице не ограничено (точнее, ограничено максимальной комбинаторной суммой вариантов комбинации имен доменов, входящих в таблицу). Уникальный ключ также называется «альтернативным», и чаще всего предназначен не для однозначной идентификации столбца, как первичный ключ, а для указания столбца, который позволяет осуществить дополнительную идентификацию строки. Например – номер паспорта, код ИНН, номер пенсионной страховки, и т.д.

В отличие от первичного ключа, в таких полях допускаются значения NULL для любого количества строк. Таким образом, можно определить ограничение уникальности для столбцов, не имеющих ограничения NOT NULL.

```

CREATE TABLE MY_TABLE (
...
FIELD1 INTEGER NOT NULL UNIQUE,
...
);

```

так и при описании таблицы (рекомендуемый вариант, с указанием имени CONSTRAINT)

```

CREATE TABLE MY_TABLE (
...
FIELD1 INTEGER NOT NULL,
FIELD2 INTEGER NOT NULL,
...
CONSTRAINT UNQ_MY_TABLE_1 UNIQUE(FIELD1, FIELD2),
...
);

```

NULL в уникальных ключах. Для уникальных ключей, содержащих несколько столбцов, логика следующая:

- во всех столбцах, входящих в уникальный ключ, нет ограничения NOT NULL;
- разрешено хранение значения NULL в столбцах, входящих в уникальный ключ, в нескольких строках;
- разрешены строки, имеющие в одном из столбцов уникального ключа значение NULL, а остальные столбцы заполнены значениями и эти значения различны хотя бы в одном из них;
- запрещены строки, имеющие в одном из столбцов уникального ключа значение NULL, а остальные столбцы заполнены значениями, и эти значения имеют совпадения хотя бы в одном из них.

Примечание. Для уникальных ключей, содержащих несколько столбцов, допускаются дубликаты значений NULL только тогда, когда они помещаются во все столбцы ограничения. В остальных случаях значения NULL будут рассматриваться как одно из обычных значений (не UNKNOWN).

```
RECREATE TABLE t(x int, y int, z int, unique(x,y,z));
INSERT INTO t values(NULL, 1, 1);
INSERT INTO t values(NULL, NULL, 1);
INSERT INTO t values(NULL, NULL, NULL);
INSERT INTO t values(NULL, NULL, NULL); -Разрешено
INSERT INTO t values(NULL, NULL, 1); -Запрещено
```

Ограничение CHECK. Ограничение CHECK задаёт условие, которому должны удовлетворять значения, помещаемые в данный столбец. Условие – это логическое выражение, называемое также предикат, которое может возвращать значения TRUE (истина), FALSE (ложь) и UNKNOWN (неизвестно). Условие считается выполненным, если предикат возвращает значение TRUE или UNKNOWN (эквивалент NULL). Если предикат возвращает FALSE, то значение не будет принято. Это условие используется при добавлении в таблицу новой строки (оператор INSERT) и при изменении существующего значения столбца таблицы (оператор UPDATE), а также операторов, в которых может произойти одно из этих действий (UPDATE или INSERT, MERGE).

Примечание. При использовании предложения CHECK для столбца, базирующегося на домене, следует помнить, что выражение в CHECK лишь дополняет условие проверки, которое может уже быть определено в домене.

На уровне столбца выражение в предложении CHECK ссылается на входящее значение с помощью ключевого слова VALUE, так же как предложение CHECK в определении домена:

```
...
CURRENCY CHAR(3) NOT NULL
CONSTRAINT CHECK_CURRENCY
CHECK (VALUE IN ('AUD', 'EUR', 'GBP', 'RUR', 'USD', 'YEN')),
...
```

На уровне таблицы выражение в предложении CHECK ссылается на входящее значение по идентификатору столбца:

```
...
CONSTRAINT CHECK_EXCHANGE
CHECK (CURRENCY IN ('AUD', 'EUR', 'GBP', 'RUR', 'USD', 'YEN')
AND CURRENCY <> EXCHANGE_CURRENCY),
...
```

1.2.2 Пример тестовых данных

Рейс

№ рейса	Пункт отправления	Пункт назначения	Время вылета	Стоимость
35	Севастополь	Москва	9-40	50
47	Севастополь	Санкт-Петербург	12-00	70
112	Москва	Санкт-Петербург	16-00	30

Полет

Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
1.10	112	15	7	Боинг-7	5
2.10	47	18	4	Боинг-7	4
3.10	35	17	3	Ту-154	2

Самолет

Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7

1.2.3 Реляционные операции

- операции над одним отношением – унарные: проекция, селекция (выборка);
- операции работы с множествами: объединение, пересечение, вычитание;
- декартово произведение и операция соединения;
- операция деления.

1.2.3.1 Проекция

Проекция – выборка из отношения данных по заданным атрибутам.



Запрос: Определить соотношение между типом самолета и допустимой массой перевозимого груза.

SQL:

```
SELECT тип_самолета, вес_груза
FROM Самолет
```

PA:

$R = \pi_{\text{тип_самолета, вес_груза}}(\text{Самолет})$

Указываем атрибуты, по которым производим проекцию на отношение

Самолет

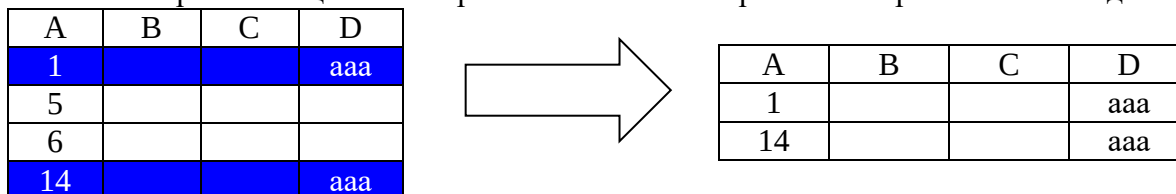
Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7

Результат

Тип самолета	Вес груза
Ту-154	5
Боинг-7	7

1.2.3.2 Селекция

Выборка-селекция – выборка из отношения кортежей с ограничением на данные.



Запрос: Вывести полную информацию о полете, которую осуществляет экипаж с кодом 17.

SQL:

```
SELECT *
FROM Полет
WHERE код_экипажа = 17
```

PA:

$R = \sigma_{\text{код_экипажа} = 17}(\text{Полет})$

Условие ограничения на выбираемые кортежи

Полет

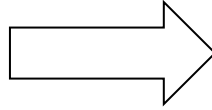
Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
1.10	112	15	7	Боинг-7	5
2.10	47	18	4	Боинг-7	4
3.10	35	17	3	Ту-154	2

Результат:

Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
3.10	35	17	3	Ту-154	2

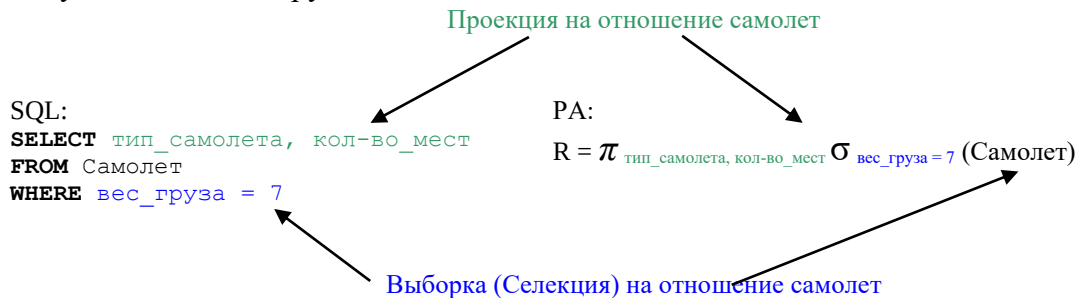
Совмещение операций Проекция и Селекция

A	B	C	D
1	a	b	10
2	c	b	12
3	d	m	3



B	C
c	b

Запрос: Вывести информацию о типе самолета и количестве мест для самолетов с допустимым весом груза 7 т.



Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7



Результат:

Тип самолета	Кол-во мест
Боинг-7	110

1.2.3.3 Примеры решения задач на PA и SQL

1. Определить число свободных мест по всем рейсам на 20.06.25.

$R = \pi_{\text{№рейса, свободные_места}} (\sigma_{\text{дата} = 20.06.25} (\text{Полет}))$;

SELECT №рейса, свободные_места
FROM Полет **WHERE** дата = "20.06.25";

2. Определить рейсы и время вылета из Симферополя в Москву.

$R = \pi_{\text{№рейса, время_вылета}} (\sigma_{\text{пункт_отправления} = \text{«Симферополь»} \wedge \text{пункт_назначения} = \text{«Москва»}} (\text{Рейс}))$;

SELECT №рейса, время вылета **FROM** Рейс
WHERE пункт_отправления = "Симферополь" **AND** пункт_назначения = "Москва";

3. Определить типы самолетов, число членов экипажа на которых 13.

$R = \pi_{\text{тип_самолета}} (\sigma_{\text{число_экипажа} = 13} (\text{Самолет}))$

SELECT тип_самолета **FROM** Самолет **WHERE** число_экипажа = 13;

1.2.3.4 Объединение

Операция *допустима только над совместными множествами-доменами* (одной природы и структуры). Т.е. операция допустима для отношений, имеющих одинаковые схемы. Если отношение имеют разные схемы, но в схемах присутствуют атрибуты, реализованные на одном домене, то операция Объединение допустима для таких отношений только при приведении этих отношений операцией проекцией (над каждым) к одинаковым схемам.

Отношение 1:

A	B	C	D

Отношение 2:

K	C	N

Шаг1. Проекция на отношение 1 по атрибуту C. Проекция на отношение 2 по атрибуту C.

C

C

Шаг 2. Объединение. Отношения имеют одинаковые схемы, операция объединения допустима.

C

Запрос: Определить тип самолета, для которых либо число членов экипажа равно 5, либо код экипажа равен 15.

Проекция: 1. приведение отношения Полет к новой схеме; 2. приведения отношения Самолет к новой схеме.

SQL:

```
SELECT тип_самолета
FROM Полет
WHERE код_экипажа = 15
UNION
SELECT тип_самолета
FROM Самолет
WHERE число_экипажа = 5
```

PA:

$R1 = \pi_{\text{тип_самолета}} \sigma_{\text{код_экипажа} = 15} (\text{Полет})$

$R2 = \pi_{\text{тип_самолета}} \sigma_{\text{число_экип} = 7} (\text{Самолет})$

$R = R1 \cup R2$

UNION – признак операции объединения для SQL.

Полет

Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
1.10	112	15	7	Боинг-7	5
2.10	47	18	4	Боинг-7	4
3.10	35	17	3	Ту-154	2

Самолет

Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7

Промежуточный результат:

Самолет:

Тип самолета
Ту-154

Полет:

Тип самолета
Боинг-7

Самолет $\cup$ Полет

Тип самолета
Ту-154
Боинг-7

1.2.3.5 Пересечение

Операция *допустима только над совместными множествами-доменами* (одной природы и структуры). Т.е. операция допустима для отношений, имеющих одинаковые схемы. Если отношение имеют разные схемы, но в схемах присутствуют атрибуты, реализованные на одном домене, то операция Пересечение допустимо для таких отношений только при приведении этих отношений операцией проекцией (над каждым) к одинаковым схемам.

Отношение 1:

A	B	C	D

Отношение 2:

K	C	N

Шаг1. Проекция на отношение 1 по атрибуту C. Проекция на отношение 2 по атрибуту C.

C

C

Шаг 2. Пересечение. Отношения имеют одинаковые схемы, операция пересечения допустима.

C

Элементы множеств, принадлежащих одновременно отношению 1 и отношению 2.

Запрос: Вывести полную информацию о полетах для самолетов с максимально допустимым весом груза до 6 т.

Неявное приведение отношения Полет к новой схеме по заданному атрибуту.

Проекция: приведение отношения Самолет к новой схеме.

SQL:
SELECT *
FROM Полет
WHERE тип_самолета **IN**
 (**SELECT** тип_самолета
FROM Самолет
WHERE вес_груза <= 6)

PA:
 $R1 = \pi_{\text{тип_самолета}}(\text{Полет})$
 $R2 = \pi_{\text{тип_самолета}} \sigma_{\text{вес_груза} \leq 6}(\text{Самолет})$
 $R = R1 \cap R2$
 Итоговое отношение:
 $R = \sigma_{\text{тип_самолета}}(\text{Полет})$

Выборка (Селекция) кортежа: вес груза до 6.

IN – признак операции пересечения.

Полет

Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
1.10	112	15	7	Боинг-7	5
2.10	47	18	4	Боинг-7	4
3.10	35	17	3	Ту-154	2

Самолет

Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7

Промежуточный результат:

Самолет:

Тип самолета
Ту-154

Полет:

Тип самолета
Ту-154
Боинг-7
Боинг-7
Ту-154



Самолет ∩ Полет

Тип самолета
Ту-154

Результат действия запроса $R = \sigma_{\text{тип_самолета} \in R}(\text{Полет})$:

Дата	№ рейса	Код экипажа	Свободные мест	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
3.10	35	17	3	Ту-154	2

1.2.3.6 Вычитание

Операция *допустима только над совместными множествами-доменами* (одной природы и структуры). Т.е. операция допустима для отношений, имеющих одинаковые схемы. Если отношение имеют разные схемы, но в схемах присутствуют атрибуты, реализованные на одном домене, то операция Вычитания допустима для таких отношений только при приведении этих отношений операцией проекцией (над каждым) к одинаковым схемам.

Отношение 1:

A	B	C	D

Отношение 2:

K	C	N

Шаг1. Проекция на отношение 1 по атрибуту C. Проекция на отношение 2 по атрибуту C.

C

C

Шаг 2. Вычитание. Отношения имеют одинаковые схемы, операция вычитания допустима.

C

Элементы множеств, принадлежащие отношению 1, и не принадлежащие отношению 2.

Запрос: Определить номера рейсов, которые не производились с даты A по дату B.

Неявное приведение отношений к новой схеме по заданному атрибуту.

SQL:

```
SELECT №_рейса
FROM Полет
WHERE №_рейса NOT EXISTS
  (SELECT №_рейса
   FROM Полет
   WHERE дата between A and B )
```

PA:

$R1 = \pi_{\text{№_рейса}} (\text{Полет})$

$R2 = \pi_{\text{№_рейса}} \sigma_{\text{дата} > A \wedge \text{дата} < B} (\text{Полет})$

$R = R1 - R2$

Выборка (Селекция) кортежа: дата с A по B.

NOT EXISTS– признак операции вычитания.

Полет

Дата	№_рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
1.10	112	15	7	Боинг-7	5
2.10	47	18	4	Боинг-7	4
3.10	35	17	3	Ту-154	2

Промежуточный результат:

Полет:

№_рейса
35
112
47
35

Полет (ограничения по дате с 1.10 по 2.10):

№_рейса
35
112
47

Результат действия запроса: Полет – Полет

№_рейса
∅ (null)

1.2.3.7 Декартово произведение

Единицей, над которой осуществляются действия при декартовом произведении, является кортеж отношения.

Пусть n_1 – количество атрибутов схемы отношения 1;

n_2 – количество атрибутов схемы отношения 2;

m_1 – количество кортежей отношения 1;

m_2 – количество кортежей отношения 2.

Для примера, соответственно:

$n_1 = 4$; $n_2 = 3$; $m_1 = 3$; $m_2 = 4$.

Отношение 1:

A	B	C	D
1			
2			
3			

Отношение 2:

K	C	N
1		
2		
3		
4		

отношение1 × отношение2

A	B	C	D	K	C	N
1	...			1		
...	...	...	...	...	...	...
1	...			4		
...	...	...	...	...	...	...
3	...			1		
...	...	...	...	...	...	...
3	...			4		

$$n = n_1 + n_2$$

$$m = m_1 +$$

Характеристика n – мощность отношения. $n = n_1 + n_2 = 7$.

Характеристика m – арность отношения $m = m_1 + m_2 = 12$.

Запрос: Определить все возможные комбинации между существующими рейсами и типами самолетов.

SQL:

SELECT *
FROM Самолет, Рейс

РА:

R = Самолет × Рейс

Комбинация признаков декартова произведения в SQL-запросе:

- выбираются все атрибуты – участвуют полностью схемы отношений;
- во фразе FROM дано перечисление таблиц;
- отсутствует условие ограничения.

Рейс

№ рейса	Пункт отправления	Пункт назначения	Время вылета	Стоимость
35	Севастополь	Москва	9-40	50
47	Севастополь	Санкт-Петербург	12-00	70
112	Москва	Санкт-Петербург	16-00	30

Самолет

Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7

Результат:

№ рейса	Пункт отправления	Пункт назначения	Время вылета	Стоимость	Тип самолета	Число экипажа	Кол-во мест	Вес груза
35	Севастополь	Москва	9-40	50	Ту-154	5	80	5
35	Севастополь	Москва	9-40	50	Боинг-7	8	110	7
47	Севастополь	Санкт-Петербург	12-00	70	Ту-154	5	80	5
47	Севастополь	Санкт-Петербург	12-00	70	Боинг-7	8	110	7
112	Москва	Санкт-Петербург	16-00	30	Ту-154	5	80	5
112	Москва	Санкт-Петербург	16-00	30	Боинг-7	8	110	7

1.2.3.8 Соединение

Операция соединения допустима только, если отношения содержат атрибуты, реализованные на одном домене.

Реализацию операции соединения можно рассматривать, как операцию декартова произведения, при наличии условия ограничения на общий атрибут отношений, по которому реализовано соединение.

Отношение 1:

A	B	C	D
a	b	1	K
a	d	7	l
c	g	10	m
f	b	14	n

Отношение 2:

K	C	N
x	7	u
y	14	t

Соединение отношения 1 с отношением 2 по атрибуту C.

A	B	C	D	K	N
a	d	7	l	x	u
f	b	14	n	y	t

Атрибут, по которому производится соединение, в итоговом отношении

указывается один раз.

Запрос: Вывести полную информацию по осуществляемым полетам и обеспечивающим эти полеты самолетам.

Перечисляются все атрибуты итогового отношения.

SQL 1 вариант:

```
SELECT Полет.*, Самолет.*
FROM Полет, Самолет
WHERE Полет.тип_самолета =
Самолет.тип_самолета
```

SQL 2 вариант:

```
SELECT Полет.*, Самолет.*
FROM Полет JOIN Самолет
ON полет.тип_самолета =
самолет.тип_самолета
```

Условие соединения.

Полет

Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
1.10	112	15	7	Боинг-7	5
2.10	47	18	4	Боинг-7	4
3.10	35	17	3	Ту-154	2

Самолет

Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7

Результат:

Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза	Число экипажа	Кол-во мест	Вес груза
1.10	35	17	12	Ту-154	3	5	80	5
1.10	112	15	7	Боинг-7	5	8	110	7
2.10	47	18	4	Боинг-7	4	8	110	7
3.10	35	17	3	Ту-154	2	5	80	5

1.2.3.9 Деление

Делимое – отношение 1, включает в себя связку атрибутов вида:

A	B	C	D
	1	3	
	2	1	
	2	2	
	2	3	
	3	2	

Делитель – отношение 2, включает один из атрибутов представленной пары В-С.

K	C	N
	1	
	2	
	3	

Частное – результат операции деления, отношение1 / отношение2.

B
2

Результат – значение по атрибуту В отношения 1, включающее множество всех допустимых значений С отношения 2.

Запрос: Определить дату, когда осуществляют рейсы все возможные типы самолетов.

Признак операции деления в запросе – все возможные.

SQL:

```
SELECT DISTINCT дата FROM Полет A
WHERE NOT EXISTS
  (SELECT тип_самолета FROM Самолет
   WHERE NOT EXISTS
     (SELECT тип_самолета FROM Полет B
      WHERE B.тип_самолета = самолет.тип_самолета
      AND A.дата=B.дата) )
```

При анализе операции деления необходимо помнить, что SQL-запросы выполняются с внутреннего уровня вложенности.

3 запрос	SELECT DISTINCT дата FROM полет A WHERE NOT EXISTS	DISTINCT исключает повторяющиеся значения при выводе результата
2 запрос	(SELECT тип_самолета FROM самолет WHERE NOT EXISTS	Формирование «дополнения» на основе существующего множества всех самолетов (отношение самолет) к множеству типов самолетов, осуществляющих полет по заданной (конкретной) дате. Если «дополнение» = $\emptyset$ – пустое множество, то в указанную дату осуществляют полет все возможные типы самолетов.
1 запрос	(SELECT тип_самолета FROM полет B WHERE B.тип_самолета = самолет.тип_самолета and A.дата=B.дата))	Формирование множества типов самолетов, осуществляющих полет в конкретную дату. Связывание двух копий таблицы полет по дате вылета.

Полет

Дата	№ рейса	Код экипажа	Свободные места	Тип самолета	Объем груза
1.10	35	17	12	Ту-154	3
1.10	112	15	7	Боинг-7	5
2.10	47	18	4	Боинг-7	4
3.10	35	17	3	Ту-154	2

Самолет

Тип самолета	Число экипажа	Кол-во мест	Вес груза
Ту-154	5	80	5
Боинг-7	8	110	7

Для реализации операции деления указанные отношения могут быть приведены к следующему виду:

Полет А

Дата	Тип самолета
1.10	Ту-154
1.10	Боинг-7
2.10	Боинг-7
3.10	Ту-154

Самолет

Тип самолета
Ту-154
Боинг-7

Полет В

Дата	Тип самолета
1.10	Ту-154
1.10	Боинг-7
2.10	Боинг-7
3.10	Ту-154

Результат операции деления:

Дата
1.10

Связывание на уровне вложенного внутреннего запроса двух копий А и В отношения полет по дате = '1.10' (первый кортеж отношения полет А). Результатом первого запроса будет множество {Ту-154, Боинг-7}.

Второй запрос работает с отношением самолет, формируя «дополнительное» множество к полученному. Для даты = '1.10' имеем $\emptyset$.

Третий запрос формирует результат. Если этот запрос выдает $\emptyset$, то в указанную дату использовались все типы самолетов.

Для даты = '1.10' (вторая строка отношения полет А): 1 запрос – {Ту-154, Боинг-7}; 2 запрос – $\emptyset$; 3 запрос не включает дату '1.10' в итоговый список повторно (работает DISTINCT).

Для даты = '2.10': 1 запрос – {Боинг-7}; 2 запрос – {Ту-154}; 3 запрос не включает дату '2.10' в итоговый список.

Для даты = '3.10': 1 запрос – {Ту-154}; 2 запрос – {Боинг-7}; 3 запрос не включает дату '3.10' в итоговый список.

1.3 Порядок выполнения лабораторной работы

1. Проанализировать описание БД по варианту (**Приложение А**), выделить и классифицировать все существующие связи, определить необходимые ограничения целостности. Построить схему БД в нотации IDEF1X.
2. Создать базу данных и все ее таблицы. Особое внимание уделить описанию первичного ключа, значений по умолчанию, описателям NOT NULL и конструкции CHECK.
3. Установить связи между таблицами.
4. Занести в таблицу образцы данных оператором INSERT INTO. **Необходимо занести не менее 15 строк.**
5. Проверить работу ограничений целостности (каскадирование удаления, модификации и др.)
6. Продемонстрировать выполнение простых вычислений в запросе.
7. Использовать простое вычисление как параметр агрегатной функции.
8. Продемонстрировать работу предложения GROUP BY.
9. Продемонстрировать работу предложения HAVING.
10. Ознакомиться с операциями реляционной алгебры.
11. Применить к разработанной БД (лабораторная работа №1) операции селекции и соединения в одном запросе.
12. Создать запрос, использующий операции проекции и деления (в одном запросе).
13. Создать запрос, использующий операции проекции, объединения и конъюнкции (в одном запросе).
14. Создать запрос, использующий операции соединения и деления (в одном запросе).
15. Создать запрос, использующий операции вычитания и дизъюнкции (в одном запросе).
16. Подготовить и защитить отчет.

1.4 Содержание отчета

Отчет состоит из титульного листа, цели работы, описания процесса выполнения работы и вывода.

Отчет должен содержать исходные данные (вариант задания), созданную схему БД, листинг создания таблиц и ограничений, **созданные (и заполненные!) таблицы, запросы должны быть сформулированы (на русском языке), представлены в форме PA и SQL.**

1.5 Контрольные вопросы

1. Что такое ограничения (constraints) в SQL? Охарактеризуйте основные типы ограничений, поддерживаемых в SQL (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK и DEFAULT).
2. Ограничения на уровне таблицы и на уровне столбца. Особенности и различия.
3. Объясните назначение PRIMARY KEY и какие требования к нему? Каковы отличия между PRIMARY KEY и UNIQUE?
4. Пример SQL-кода для установки ограничения.
5. Объясните назначение внешнего ключа и как он связывает таблицы? Как ограничение FOREIGN KEY влияет на операции DELETE и UPDATE?
6. Как можно предотвратить удаление записи из родительской таблицы, когда существуют связанные записи в дочерней таблице? Что означает действия CASCADE, SET NULL, SET DEFAULT, и NO ACTION, и как они работают?
7. Приведите классификацию операций PA и перечислите их.
8. Поясните действие операций реляционной алгебры с примерами.
9. Приведите пример операции селекции.
10. Чем отличаются операции PA соединение и объединение.

11. Продемонстрируйте на примере, как выразить операцию соединения через декартово произведение.
12. Сформулировать и записать запрос на РА, не реализующийся на SQL.

ПРИЛОЖЕНИЕ А**Варианты заданий****Вариант 1: База данных заказов**

Клиенты (id_клиента, имя, фамилия, адрес, телефон, электронная_почта, дата_регистрации)

Заказы (id_заказа, id_клиента, дата_заказа, статус, общая_сумма, id_способа_оплаты)

Товары (id_товара, название, описание, цена, категория, производитель, дата_выпуска)

Способы_оплаты (id_способа_оплаты, название, комиссия)

Поставщики (id_поставщика, имя, контактные_данные)

Заказанные_товары (id_товара, id_заказа, количество, цена_заказа)

Вариант 2: База данных продуктов с отзывами

Продукты (id_продукта, название, описание, цена, категория, производитель, фото)

Категории (id_категории, название, родительская_категория, описание)

Запасы (id_продукта, количество_в_наличии, минимальный_порог, дата_обновления)

Отзывы (id_отзыва, id_продукта, id_пользователя, рейтинг, текст_отзыва, дата_отзыва)

Вариант 3: База данных электронной коммерции

Пользователи (id_пользователя, имя, фамилия, адрес, телефон, электронная_почта, дата_регистрации, уровень_доступа)

Продукты (id_продукта, название, описание, цена, категория, производитель, рейтинг)

Заказы (id_заказа, id_пользователя, дата_заказа, статус, общая_сумма, id_сервиса_доставки)

Сервисы_доставки (id_сервиса_доставки, название, стоимость, срок_доставки)

Картинки_продуктов (id_продукта, url_изображения)

Вариант 4: База данных сотрудников с проектами

Сотрудники (id_сотрудника, имя, фамилия, должность, зарплата, дата_приема, id_менеджера)

Отделы (id_отдела, название, описание, ответственное_лицо)

Проекты (id_проекта, название, описание, дата_старта, дата_окончания, статус)

Участие (id_сотрудника, id_проекта, роль, дата_начала, дата_окончания, комментарии)

Вариант 5: База данных библиотеки

Книги (id_книги, название, автор, жанр, ISBN, дата_публикации, количество_страниц)

Авторы (id_автора, имя, фамилия, дата_рождения, биография)

Читатели (id_читателя, имя, фамилия, адрес, телефон, дата_регистрации)

Выдачи (id_выдачи, id_книги, id_читателя, дата_выдачи, дата_возврата, штраф)

Группы_читателей (id_группы, название, описание, льготы)

Вариант 6: База данных больницы

Пациенты (id_пациента, имя, фамилия, дата_рождения, пол, адрес, телефон)

Врачи (id_врача, имя, фамилия, специальность, рейтинг, телефон)

Приемы (id_приема, id_пациента, id_врача, дата_приема, диагноз, рекомендации)

Лабораторные_анализы (id_анализа, id_приема, название, результат, дата_анализа)

Вариант 7: База данных школы с учебными планами

Ученики (id_ученика, имя, фамилия, класс, дата_рождения, дата_поступления)

Учителя (id_учителя, имя, фамилия, предмет, стаж, телефон, email)

Классы (id_класса, название, год_обучения, id_учителя)

Оценки (id_оценки, id_ученика, id_предмета, оценка, дата)

Предметы (id_предмета, название, учебный_план)

Вариант 8: База данных выставок недвижимости

Недвижимость (id_недвижимости, тип, адрес, площадь, цена, состояние, год_постройки)
 Типы_недвижимости (id_типа, название)
 Агенты (id_агента, имя, фамилия, телефон, e-mail, рейтинг)
 Выставки (id_выставки, id_недвижимости, дата, место, количество_записей)
 Продажи (id_продажи, id_недвижимости, id_агента, дата_продажи, цена)

Вариант 9: База данных туристического агентства

Направления (id_направления, название, страна, описание, популярность)
 Отели (id_отеля, название, адрес, звездность, рейтинг, услуги)
 Полеты (id_полета, номер_рейса, дата_отправления, дата_прибытия, цена, авиакомпания)
 Бронирования (id_бронирования, id_направления, id_отеля, id_полета, имя, фамилия, дата_бронирования, статус)
 Отзывы (id_отзыва, id_бронирования, текст_отзыва, оценка, дата_отзыва)

Вариант 10: База данных финансовых операций

Счета (id_счета, номер_счета, баланс, тип_счета)
 Транзакции (id_транзакции, id_счета, дата_транзакции, сумма, тип, комментарий, категория)
 Категории (id_категории, название, описание)
 Бюджеты (id_бюджета, id_категории, ежемесячный_лимит, фактические_расходы)
 Счета_для_сбережений (id_счета, цель, текущий_баланс, рекомендуемая_сумма)

Вариант 11: База данных социальной сети

Пользователи (id_пользователя, имя, фамилия, электронная почта, дата_регистрации, биография)
 Посты (id_поста, id_пользователя, дата_публикации, текст, количество_лайков, количество_комментариев)
 Подписчики (id_подписки, id_подписчика, id_подписки, дата_подписки)
 Комментарии (id_комментария, id_поста, id_пользователя, текст, дата)
 Группы (id_группы, название, описание, создающий, дата_создания)

Вариант 12: База данных концертной деятельности артиста

Альбомы (id_альбома, название, исполнитель, год_выпуска, жанр, количество_песен)
 Исполнители (id_исполнителя, имя, жанр, биография)
 Песни (id_песни, название, id_альбома, продолжительность, текст)
 Концерты (id_концерта, дата, место, id_исполнителя, стоимость)
 Билеты (id_билета, id_концерта, категория, цена, количество_доступных)

Вариант 13: База данных спортивных мероприятий

Спортсмены (id_спортсмена, имя, фамилия, вид_спорта, страна, дата_рождения, рейтинг)
 События (id_события, название, дата, место_проведения, тип_соревнования)
 Результаты (id_результата, id_спортсмена, id_события, место, время, очки)
 Команды (id_команды, название, вид_спорта, страна)
 Тренеры (id_тренера, имя, фамилия, специализация, стаж)

Вариант 14: База данных гидрометцентра

Города (id_города, название, страна, население)
 Даты (id_даты, дата, день_недели, месяц)
 Температура (id_температуры, id_города, id_даты, дневная, ночная, средняя)
 Осадки (id_осадков, id_города, id_даты, тип, количество, вероятность)
 Климат (id_климата, id_города, тип_климата, характеристики)

Вариант 15: База данных растений и их уходе

Растения (id_растения, название, тип, семейство, стоимость, место_происхождения)

Типы растений (id_типа, название, описание)
 Семейства (id_семейства, название, особенности)
 Характеристики (id_характеристики, id_растения, особенность, значение)
 Уход (id_ухода, id_растения, дата, операция, комментарий)

Вариант 16: База данных автосалона

Автомобили (id_автомобиля, марка, модель, год_выпуска, цена, цвет, тип_трансмиссии, пробег)
 Покупатели (id_покупателя, имя, фамилия, адрес, телефон, электронная_почта)
 Заказы (id_заказа, id_покупателя, id_автомобиля, дата_заказа, статус, сумма)
 Сервисы (id_сервиса, название, адрес, телефон, виды_услуг)
 Обслуживание (id_обслуживания, id_автомобиля, id_сервиса, дата, описание, стоимость)

Вариант 17: База данных для системы бронирования билетов на мероприятия

Мероприятия (id_мероприятия, название, дата, время, место, описание, статус)
 Участники (id_участника, имя, фамилия, электронная_почта, телефон, организацией)
 Бронирования (id_бронирования, id_мероприятия, id_участника, количество_билетов, стоимость_билета)
 Платежи (id_платежа, id_бронирования, дата_платежа, сумма, способ_оплаты, статус)
 Спонсоры (id_спонсора, название, контактный_менеджер, телефон, email, сумма_взноса)

Вариант 18: База данных онлайн-курсов

Курсы (id_курса, название, описание, стоимость, дата_начала, дата_окончания, преподаватель)
 Студенты (id_студента, имя, фамилия, электронная_почта, телефон, дата_регистрации)
 Записи (id_записи, id_студента, id_курса, статус, дата_записи)
 Материалы (id_материала, id_курса, название, тип, ссылка, дата_добавления)
 Оценки (id_оценки, id_записи, баллы, комментарий, дата)

Вариант 19: База данных фитнес-клуба

Клиент (id_клиента, имя, фамилия, дата_рождения, телефон, дата_подписки, тип_членства)
 Тренеры (id_тренера, имя, фамилия, специализация, опыт, телефон)
 Занятия (id_занятия, название, расписание, длительность, id_тренера)
 Записи на занятия (id_записи, id_клиента, id_занятия, дата_записи, статус)
 Услуги (id_услуги, название, описание, стоимость)

Вариант 20: База данных телеконференций

Конференции (id_конференции, название, дата, время, место, описание)
 Докладчики (id_докладчика, имя, фамилия, организация, тема, био)
 Участники (id_участника, имя, фамилия, электронная_почта, статус_участия)
 Регистрации (id_регистрации, id_участника, id_конференции, статус, дата)
 Сессии (id_сессии, id_конференции, название, начало, окончание)

Вариант 21: База данных онлайн-игр

Игры (id_игры, название, жанр, дата_выпуска, платформа, разработчик, рейтинг)
 Пользователи (id_пользователя, имя, фамилия, электронная_почта, дата_регистрации)
 Игровые сессии (id_сессии, id_пользователя, id_игры, дата, время, результат)
 Отзывы (id_отзыва, id_игры, id_пользователя, текст, оценка, дата_отзыва)
 Достижения (id_достижения, id_игры, название, описание)

Вариант 22: База данных для гостиничного бизнеса

Гостиницы (id_гостиницы, название, адрес, телефон, рейтинг, описание)
 Комнаты (id_комнаты, id_гостиницы, номер, тип, цена, состояние)

Бронирования (id_бронирования, id_гостиницы, id_комнаты, имя_гостя, дата_заезда, дата_выезда, статус)

Услуги (id_услуги, id_гостиницы, название, описание, стоимость)

Отзывы (id_отзыва, id_гостиницы, имя_гостя, текст, рейтинг, дата)

Вариант 23: База данных для ресторанного бизнеса

Рестораны (id_ресторана, название, адрес, телефон, рейтинг, кухня)

Меню (id_блюда, id_ресторана, название, описание, цена, тип_блюда)

Заказы (id_заказа, id_ресторана, дата_заказа, имя_клиента, статус, общая_сумма)

Отзывы (id_отзыва, id_ресторана, имя_клиента, текст, дата, рейтинг)

Специальные_предложения (id_предложения, id_ресторана, название, описание, дата_начала, дата_окончания)

Вариант 24: База данных для управления сессиями веб-приложения

Пользователи (id_пользователя, имя, фамилия, электронная_почта, пароль, дата_регистрации)

Сессии (id_сессии, id_пользователя, время_входа, время_выхода, ip_адрес)

Действия (id_действия, id_сессии, время_действия, тип_действия, параметры)

Устройства (id_устройства, id_пользователя, тип, операционная_система, браузер)

Список паролей (id_логина, id_пользователя, старая_версия, новая_версия, дата_смены)

Вариант 25: База данных для управления проектами и задачами

Проекты (id_проекта, название, дата_начала, дата_окончания, статус, описание)

Задачи (id_задачи, id_проекта, название, описание, статус, дата_сдачи, приоритет)

Участники (id_участника, имя, фамилия, email, роль)

Баг-репорт (id_записи, id_задачи, причина, дата)

Комментарии (id_комментария, id_задачи, id_участника, текст, дата)

Вариант 26: База данных читального зала

Книги (id_книги, название, автор, год_издания, ISBN, жанр, стоимость)

Авторы (id_автора, имя, фамилия, дата_рождения, биография, список_книг)

Читатели (id_читателя, имя, фамилия, адрес, телефон, дата_записи)

Выдачи (id_выдачи, id_книги, id_читателя, дата_выдачи, дата_возврата, статус)

Отзывы (id_отзыва, id_книги, id_читателя, текст, оценка, дата)

Вариант 27: База данных для отслеживания качества продукта

Продукты (id_продукта, название, производитель, категория, стоимость)

Поставщики (id_поставщика, название, контактные_данные, адрес)

Проверки_качества (id_проверки, id_продукта, дата_проверки, статус, заключение)

Неисправности (id_неисправности, id_проверки, описание, дата_выявления)

Реакции_на_неисправности (id_реакции, id_неисправности, тип_реакции, дата_реакции)

Вариант 28: База данных для управления магазинами

Магазины (id_магазина, название, адрес, телефон, тип_магазина)

Товары (id_товара, id_магазина, название, категория, цена, количество_в_наличии)

Заказы (id_заказа, id_магазина, id_покупателя, дата, статус, общая_сумма)

Поставщики (id_поставщика, название, контактные_данные)

Скидки (id_скидки, id_товара, процент, дата_начала, дата_окончания)

Вариант 29: База данных для управления процессами производства

Продукция (id_продукции, название, тип, цена, количество)

Сырьё (id_сырья, название, цена, количество_в_наличии)

Процессы (id_процесса, название, рабочая_группа, время_выполнения, статус)

Рабочие (id_рабочего, имя, фамилия, id_процесса, опыт)

Отчёты (id_отчета, id_процесса, дата, проблемы, успешность)

Вариант 30: База данных курсов изучения языков

Языки (id_языка, название, уровень_сложности, количество_учеников)

Курсы (id_курса, id_языка, название, описание, стоимость, дата_начала, дата_окончания, преподаватель)

Студенты (id_студента, имя, фамилия, электронная_почта, уровень_знаний)

Записи (id_записи, id_студента, id_курса, дата_записи, статус)

Оценки (id_оценки, id_записи, баллы, дата, комментарии)

Вариант 31: База данных онлайн-записи

Курсы (id_курса, название, описание, стоимость, дата_начала, дата_окончания, id_преподавателя)

Преподаватели (id_преподавателя, имя, фамилия, электронная_почта, специализация)

Студенты (id_студента, имя, фамилия, электронная_почта, дата_регистрации)

Записи на курсы (id_записи, id_студента, id_курса, дата_записи, статус_записи)

Материалы курса (id_материала, id_курса, название, тип_материала, ссылка)

Отзывы (id_отзыва, id_курса, id_студента, текст, оценка, дата_отзыва)